Formal Methods in Software Engineering

Theory of Computation, Spring 2026

Konstantin Chukharev

Languages

Why Theory of Computation?

We have studied *propositional logic*, *SAT*, and *first-order logic*.
A natural question recurs: **"Which problems can be solved automatically?"**

The central question of this lecture: Given a decision problem (*e.g.*, "is this FOL formula valid?"), does there exist an *algorithm* that always answers correctly in *finite time*?

We have already seen:

- SAT is NP-complete — hard, but *decidable (an answer always exists, just slowly)*
- FOL validity is only *semi-decidable (provability but not refutability)*
- Some SMT theories are *decidable (e.g., linear arithmetic over $\mathbb{R}$)*
- FOL over $\mathbb{N}$ (Peano Arithmetic) is undecidable

This lecture provides:

- Formal definition of "computation"
- Precise meaning of *decidability*
- Turing machines as a model of computation
- Rice's theorem: why program verification is *hard in general*
- Why SMT solvers restrict to specific *theories*

Formal Languages

Definition 1: An *alphabet* Σ is a finite non-empty set of symbols.

A *word* (or *string*) over Σ is a finite sequence of symbols from Σ. The *empty word* is ε.

The set of all finite words over Σ is $\Sigma^* = \bigcup_{k=0}^{\infty} \Sigma^k$.

A *formal language* $L \subseteq \Sigma^*$ is any set of finite words over Σ.

Example:

- $\Sigma = \{0, 1\}$, $L_1 = \{0^n 1^n \mid n \geq 0\} = \{\varepsilon, 01, 0011, 000111, \ldots\}$
- $\Sigma = \{a, b\}$, $L_2 = \{w \mid w \text{ has equal number of } a\text{s and } b\text{s}\}$
- $L_3 = \{\mathrm{SAT}, \mathrm{HALT}, \mathrm{VALID}, \ldots\}$ — languages encoding decision problems

Note: Every decision problem is a formal language: the set of *yes-instances*. Solving the problem = deciding membership in the language.

Chomsky Hierarchy

Formal languages are classified into four nested levels:

Type	Class	Recognizing Machine	Example Language
3	Regular	DFA / NFA	a^*b^*, $\{a^n \mid n \text{ even}\}$
2	Context-Free	Pushdown Automaton	$\{a^n b^n \mid n \geq 0\}$
1	Context-Sensitive	Linear-Bounded TM	$\{a^n b^n c^n \mid n \geq 0\}$
0	Recursively Enumerable	Turing Machine	$\{\langle M, w\rangle \mid M \text{ halts on } w\}$

Each level adds *more memory*: finite states → unlimited stack → bounded tape → infinite tape.

More expressive = harder algorithmic questions about the language class.

Note: The classes are *nested*: every regular language is context-free, every context-free is context-sensitive, *etc.* The containments are strict — each level is strictly more powerful than the one below.

Decision Problems as Languages

Definition 2: A *decision problem* is a question with a "yes" or "no" answer depending on the input.

Formally, the set of inputs for which the answer is "yes" forms a language $L \subseteq \Sigma^*$.

Deciding the problem = *recognizing* the language L.

SAT: Given a CNF formula φ, is it satisfiable?

$$\text{SAT} = \{\varphi \mid \varphi \text{ is a satisfiable Boolean formula}\}$$

FOL Validity: Given a first-order formula φ, is it valid?

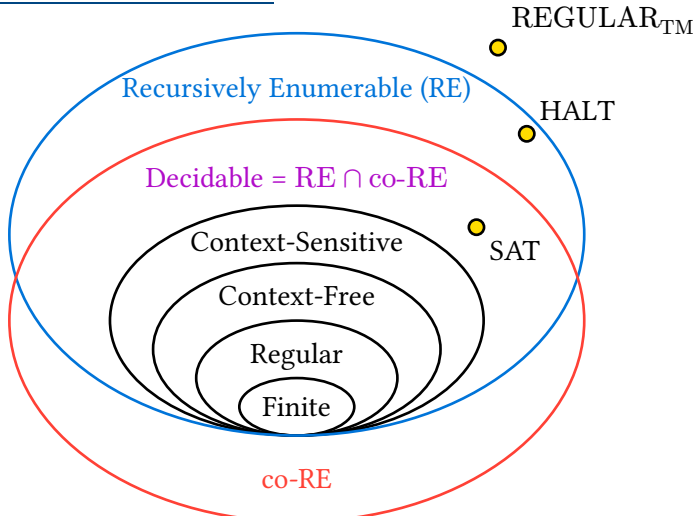
$$\text{VALID} = \{\varphi \mid \varphi \text{ is a valid (universally true) FOL formula}\}$$

Halting Problem: Given a TM M and input w, does M halt on w?

$$\text{HALT} = \{\langle M, w\rangle \mid \text{TM } M \text{ halts on input } w\}$$

"Is $w \in L$?" and "does the algorithm say yes on input w?" are *the same question*.

Language Complexity Classes



Note: **SAT** is decidable (NP-complete). **HALT** is recognizable but *not* decidable: a TM can confirm halting by simulation, but cannot confirm non-halting. REGULAR$_{\text{TM}}$ = "does TM M recognize a regular language?" — in neither RE nor co-RE.

Machines

Finite Automata

Definition 3: A *Deterministic Finite Automaton* (DFA) is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$:

- Q — finite set of *states*
- Σ — *input alphabet*
- $\delta : Q \times \Sigma \to Q$ — *transition function*
- $q_0 \in Q$ — *start state*
- $F \subseteq Q$ — set of *accepting states*

A DFA processes input left-to-right, one symbol at a time, and accepts if it ends in an accepting state. DFAs recognize exactly the *regular* languages (Type 3 in the Chomsky hierarchy).

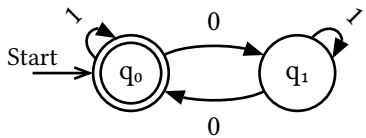
DFA Example: Even Number of 0s

Example: Automaton $\mathcal{A}$ over $\Sigma = \{0, 1\}$ recognizing $\mathcal{L}(\mathcal{A}) = \{w \mid w \text{ has an even number of 0s}\}$.

States: q_0 = "seen even many 0s" (start, accepting), q_1 = "seen odd many 0s".

- Reading a **0**: flip parity (switch state).
- Reading a **1**: parity unchanged (stay in state).

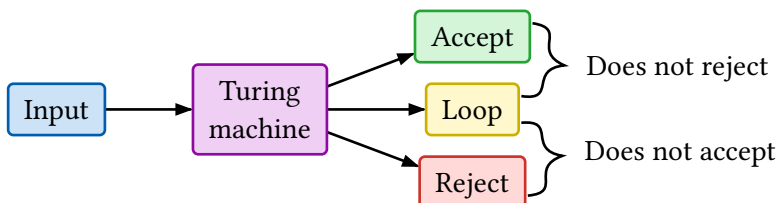
	0	1
q0	q1	q0
q1	q0	q1



Turing Machines

Informally, a Turing machine is a *finite-state* machine with an *infinite tape* and a *head* that can read and write symbols. Initially, the tape contains the *input* string, the rest are blanks, and the machine is in the *start* state. At each step, the machine reads the symbol under the head, changes the state, writes a new symbol, and moves the head left or right. When the machine reaches the *accept* or *reject* state, it immediately halts.

Note: If the machine never reaches the *accept* or *reject* state, it *loops* forever.



TM Formal Definition

Definition 4: Turing Machine (TM) is a 7-tuple $\left(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{acc}}, q_{\text{rej}}\right)$ where:

- Γ is a *tape alphabet* (including blank symbol $\square \in \Gamma$),
- $\Sigma \subseteq \Gamma$ is a *input alphabet*,
- $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ is a transition function,
- q_{acc} and q_{rej} are the *accept* and *reject* states.

TM recognizes *recursively enumerable* languages (Type 0).

TM Language and Acceptance

Definition 5: The language *recognized* by M, written $\mathcal{L}(M)$, is the set of inputs M accepts:

$$\mathcal{L}(M) = \{w \in \Sigma^* \mid M \text{ halts in state } q_{\text{acc}} \text{ on input } w\}$$

For inputs *not* in $\mathcal{L}(M)$, the machine either *rejects* (halts in q_{rej}) or *loops forever*.

- A *recognizer* only needs to accept members of L. It may loop forever on non-members.
- A *decider* must always halt — it accepts members and *rejects* non-members.

$$\text{Decider} = \text{Recognizer that never loops}$$

Definition 6: A TM is a *decider* for L if it halts on *every* input (accepting L and rejecting $\overline{L}$).

- A language is *decidable* (recursive) if it has a decider.
- A language is *recognizable* (RE) if it has a recognizer.

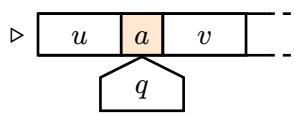
TM Configuration

Definition 7: A *configuration* describes the complete state of a TM at a given moment:

$$(u; q; v) \quad u, v \in \Gamma^*, \quad q \in Q$$

- u — tape contents to the *left* of the head
- q — current *state*
- v — tape contents from the head *rightward* (head reads $v[0]$)

Example: Configuration $(u; q; av)$ is visualized as:



The head reads a; the next transition depends on (q, a).

TM Computation

Definition 8: A *computation* of TM M on input w is a sequence of configurations:

$$C_1 \Rightarrow C_2 \Rightarrow \cdots \Rightarrow C_n$$

- $C_1 = (\triangleright; q_0; w)$ — *start configuration*
- $C_i \Rightarrow C_{\{i+1\}}$ — "C_i yields $C_{\{i+1\}}$ in one step"
- C_n is a *halting configuration* (state is q_{acc} or q_{rej})

The relation $\Rightarrow^*$ (yields in any number of steps) is the reflexive-transitive closure of $\Rightarrow$.

Intuition: Think of a computation as a "snapshot sequence" of the machine. Each snapshot captures the tape contents, the current state, and the head position. The machine moves from snapshot to snapshot by applying one transition.

Note: A *terminating* computation always reaches q_{acc} or q_{rej}. A *looping* computation produces an infinite sequence $C_1 \Rightarrow C_2 \Rightarrow \cdots$ that never halts.

TM Yields Relation

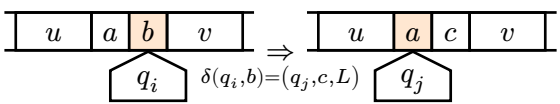
How does one configuration yield the next?

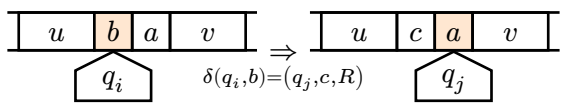
Definition 9: Let $u, v \in \Gamma^*$, $a, b, c \in \Gamma$, $q_i, q_j \in Q$.

Move left (L): $(ua; q_i; bv) \Rightarrow \big(u; q_j; acv\big)$ when $\delta(q_i, b) = \big(q_j, c, L\big)$

Move right (R): $(u; q_i; bav) \Rightarrow \big(uc; q_j; av\big)$ when $\delta(q_i, b) = \big(q_j, c, R\big)$

In both cases: overwrite b with c, move the head, change to state q_j.

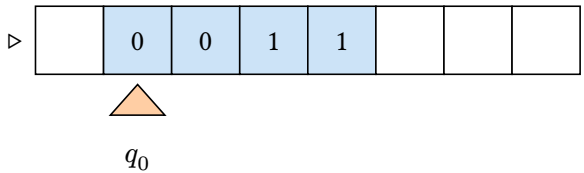




Note: **Left-end special case:** If the head is at the tape start and the transition says "move left", the head stays in place: $(\triangleright; q_i; bv) \Rightarrow \big(\triangleright; q_j; cv\big)$ when $\delta(q_i, b) = \big(q_j, c, L\big)$.

TM Tape Visualization

Example: Initial tape state for $\{0^n 1^n\}$ TM on input $w = 0011$:



Blue cells = input. The head (orange triangle) points at the first cell. State is q_0.

Note: The machine strategy: repeatedly find the leftmost `0`, mark it `X`, find the matching `1`, mark it `Y`. Accept when all 0s and 1s are paired. Reject if counts mismatch.

TM Example: Recognizing $0^n 1^n$

Step-by-step configuration trace for input 0011 ($n = 2$).

Step	**Configuration** $(u; q; v)$	**Action**
Start	$(\triangleright; q_0; 0011)$	Read `0` $\longrightarrow$ write `X`, move right
Mark first 0	$(\triangleright\ X; q_1; 011)$	Scan right past 0s to find `1`
Found first 1	$(\triangleright\ X0; q_2; 11)$	Write `Y`, move left
Mark first 1	$(\triangleright\ X0Y; q_3; 1)$	Move back to start
Back at start	$(\triangleright; q_0; X0Y1)$	Read `X` $\longrightarrow$ skip, find next `0`
Mark second 0	$(\triangleright\ XX; q_1; Y1)$	Scan right past Y to find `1`
Found second 1	$(\triangleright\ XXY; q_2; 1)$	Write `Y`, move left
All matched	$(\triangleright\ XXYY; q_{\text{acc}}; \square)$	Tape is all X/Y — **Accept!**

Key insight: The tape acts as scratch memory. At each round, one `0`–`1` pair is matched and "consumed" by overwriting with `X` and `Y`. This requires $O(n^2)$ steps for input length $2n$.

Machine Comparison

Property	DFA	PDA	TM
Memory	None (finite states)	Stack (LIFO)	Infinite R/W tape
Reading	Left-to-right, each symbol once	Left-to-right, each symbol once	Arbitrary R/W movement
Language class	Regular	Context-Free	RE (or R if decider)
Determinism	Equivalent to NDFA	NDPDA more powerful	NTM = DTM
Emptiness check	Decidable	Decidable	Undecidable
Equality check	Decidable	Undecidable	Undecidable
Example language	$a^n (n \text{ even})$	$a^n b^n$	$a^n b^n c^n$

Note: NTM = DTM is the *Church–Turing thesis* in action: non-determinism does not add power, only speed. Compare with "NP vs P"!

TM Variants

All the following are equivalent in computational power (they recognize the same class of languages):

Standard TM

- Single infinite tape, one head
- Alphabet Γ, states Q, transition δ

Multi-tape TM

- k tapes, k heads moving independently
- Easier to program, same power

Non-deterministic TM (NTM)

- At each step, choose from multiple transitions
- Accepts if *any* branch accepts
- Non-determinism $\Rightarrow$ exponential simulation

Two-way infinite tape

- Tape extends in both directions
- Simulated by storing two tapes on one

TM with stay

- Head can stay in place (S move)
- Easier to define, trivially equivalent

Random Access Machine (RAM)

- Memory indexed by address
- Polynomially equivalent to standard TM (relevant for complexity!)

Church–Turing thesis: Any reasonable model of computation computes exactly the same class of functions as a Turing machine. The thesis is supported by the equivalence of all known models.

Recognizing vs Deciding

There are *two* types of Turing machines:

1. **Decider** (total TM): always halts on every input.
2. **Recognizer** (general TM): may loop forever on some inputs.

Definition 10 (Recognition): A TM *recognizes* language L if it accepts every $w \in L$ and does not accept any $w \notin L$ (but may loop on non-members).

Such a language is called *recognizable* (also: Turing-recognizable, recursively enumerable, semi-decidable — all equivalent). The class of all recognizable languages is **RE**.

Definition 11 (Decision): A TM *decides* language L if it accepts every $w \in L$ and *rejects* every $w \notin L$; it always halts.

Such a language is called *decidable* (also called: recursive, computable). The class of all decidable languages is **R** ($\subsetneq$ RE).

Recognizing vs Deciding [2]

Key distinction:

- A recognizer is allowed to loop on non-members.
- A decider *must* halt and give an answer for every input.

MIU. MU?

Definition 12 (MIU system): The *MIU system* is a "formal system" consisting of:

- an alphabet $\Sigma = \{\mathtt{M}, \mathtt{I}, \mathtt{U}\}$,
- a single axiom: `MI`,
- a set of inference rules:

Rule	Description	Example
$x\mathtt{I} \vdash x\mathtt{IU}$	add `U` to the end of any string ending with `I`	`MI` to `MIU`
$\mathtt{M}x \vdash \mathtt{M}xx$	double the string after M	`MIU` to `MIUIU`
$x\mathtt{III}y \vdash x\mathtt{U}y$	replace any `III` with `U`	`MUIIIU` to `MUUU`
$x\mathtt{UU}y \vdash xy$	remove any UU	`MUUU` to `MU`

Question: Is `MU` a theorem of the MIU system?

Complexity

P and NP

Definition 13: Class P consists of problems that can be *solved* in *polynomial time*.

Equivalently, $L \in P$ iff L is *decidable* in polynomial time by a *deterministic* TM.

Examples: Shortest path, primality testing (AKS algorithm), linear programming.

Definition 14: Class NP consists of problems where a *certificate* of a solution ("yes" answer) can be *verified* in polynomial time.

Equivalently, $L \in \mathrm{NP}$ iff L is *decidable* in polynomial time by a *non-deterministic* TM.

Equivalently, $L \in \mathrm{NP}$ iff L is *recognizable* in polynomial time by a *deterministic* TM.

Examples: SAT, graph coloring, graph isomorphism, subset sum, knapsack, vertex cover, clique.

NP-Hard and NP-Complete

Definition 15: A problem H is *NP-hard* if every problem $L \in \mathrm{NP}$ is polynomial-time *reducible* to H.

Examples: Halting problem (undecidable), Traveling Salesman Problem (TSP).

Definition 16: A problem H is *NP-complete* if:

1. $H \in \mathrm{NP}$
2. H is NP-hard

Examples: SAT, 3-SAT, Hamiltonian path... Actually, almost all NP problems are NP-complete!

Theorem 1 (Cook–Levin): SAT is NP-complete.

co-NP

Definition 17: Complexity class co-NP contains problems where *"no"* instances can be *verified* in *polynomial time*.

Equivalently, $L \in$ co-NP iff the complement of L is in NP:

$$\text{co-NP} = \left\{ L \mid \overline{L} \in \text{NP} \right\}$$

Open question: NP $\stackrel{?}{=}$ co-NP? Implies P $\neq$ NP if false.

Examples:

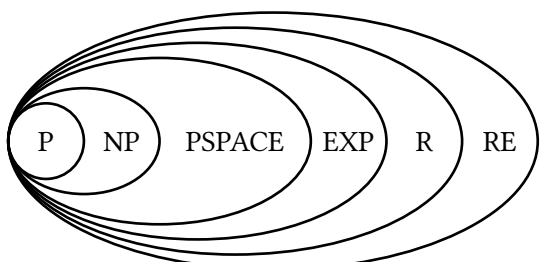
- **VALID**: Check if a Boolean formula is always true (tautology).
- **UNSAT**: Check if a formula has no satisfying assignment.

Computational Hierarchy

$\mathrm{P} \subseteq \mathrm{NP} \subseteq \mathrm{PSPACE} \subseteq \mathrm{EXP} \subset \mathrm{R} \subset \mathrm{RE}$

- **RE** — Languages *accepted* by any TM. Not all RE languages are decidable.
- **R** = RE ∩ co-RE — Languages *decided* by a halting TM. Closed under complement.
- **EXP** — Decided in *exponential time* by a deterministic TM. Closed, proper superset of NP.
- **PSPACE** — Decided in *polynomial space*. Contains NP and co-NP. QBF is PSPACE-complete.
- **NP** — Accepted by a *non-deterministic* TM in polynomial time. SAT, graph coloring, *etc.*
- **P** — Decided in *polynomial time* by a deterministic TM. Primality, BFS/DFS, LP.

Note: All containments are known, but many strict separations are open: $\mathrm{P} \stackrel{?}{=} \mathrm{NP}$, $\mathrm{NP} \stackrel{?}{=} \text{co-NP}$, $\mathrm{NP} \stackrel{?}{=} \mathrm{PSPACE}$, *etc.*



Polynomial Hierarchy

The *polynomial hierarchy* PH refines the NP/co-NP picture using alternating quantifiers:

$$\Sigma_0^P = \Pi_0^P = \mathrm{P}$$

$$\Sigma_{k+1}^P = \mathrm{NP}^{\Sigma_k^P}, \quad \Pi_{k+1}^P = \text{co-NP}^{\Sigma_k^P}$$

- $\Sigma_1^P = \mathrm{NP}$: $\exists$ witness, polynomial verifier
- $\Pi_1^P = \text{co-NP}$: $\forall$ witnesses, polynomial verifier
- Σ_2^P: $\exists\forall$ witnesses (*e.g.*, "does φ have an assignment that satisfies all clauses for every setting of some variables?")
- Π_2^P: $\forall\exists$ witnesses

- Model checking for μ-calculus is PSPACE-complete.
- Bounded model checking for quantified Boolean formulas (QBF) is PSPACE-complete.
- SMT with quantifiers ($\begin{smallmatrix}\forall\\ \exists\end{smallmatrix}$) stratifies across the polynomial hierarchy.

Note: If PH collapses to any level (*i.e.*, $\Sigma_k^P = \Sigma_{k+1}^P$ for some k), it implies strong consequences about the structure of NP. In particular, $\mathrm{P} = \mathrm{NP}$ would collapse the entire hierarchy to P.

Complexity Zoo

Problem	Complexity	Practical Approach
Propositional SAT	NP-complete	CDCL SAT solvers (CaDiCaL, MiniSat)
QBF (SMT over 2nd order)	PSPACE-complete	QBF solvers (DepQBF)
Linear arith. ($\mathbb{R}$)	P (LP), NP in SMT	Simplex + DPLL(T) (Z3, CVC5)
Linear arith. ($\mathbb{Z}$)	NP-hard	Branch-and-bound + cutting planes
Non-linear arith. ($\mathbb{R}$)	Decidable (Tarski)	Cylindrical Algebraic Decomp.
Non-linear arith. ($\mathbb{Z}$)	Undecidable (Hilbert 10)	Semi-decidable fragments only
FOL validity	Undecidable (semi-decidable)	Tableau / resolution (incomplete)
Program verification	Undecidable (Rice)	Require invariants, bounded checking

Why decidable fragments matter: SMT works precisely because it uses *restricted but decidable* theories. Once you add full integer multiplication or quantified arithmetic, the theories become undecidable — and no complete solver can exist.

See also: Complexity Zoo Petting Zoo

Computability

The Church–Turing Thesis

Church–Turing Thesis: Every *effectively computable* function is computable by a Turing machine.

Note: "Effectively computable" means: can be carried out by a *finite*, *deterministic*, *mechanical* step-by-step *procedure*, with no creativity or *luck* required.

Note: This is a **thesis**, not a theorem, since the notion "effectively computable" is informal and cannot be precisely defined. We cannot formally *prove* the thesis, but no counterexample has ever been found.

Historical note: In 1936, Alonzo Church (λ-calculus) and Alan Turing (Turing machines) independently formalized computability and proved these models equivalent.

Every other general-purpose model proposed since — register machines, μ-recursive functions, Post systems, Markov normal algorithms — computes exactly the same class of functions.

Computable Functions

Definition 18 (Computable function): A partial function $f : \mathbb{N}^k \hookrightarrow \mathbb{N}$ is *computable* if there exists a TM M such that:

- If $f(\vec{x})$ is defined: M halts on input $\vec{x}$ with output $f(\vec{x})$.
- If $f(\vec{x})$ is undefined: M loops forever on input $\vec{x}$.

Computable functions:

- $f(x) = x^2$, $f(x) = x!$, $f(x) = x \bmod 2$ — basic arithmetic
- $f(n) =$ the n-th prime — search computable
- $f(n) =$ the n-th digit of π — BBP formula
- The Ackermann function $A(m, n)$ — computable but not primitive recursive

Non-computable functions:

Definition 19 (Busy Beaver): $\mathrm{BB}(n)$ = the maximum number of 1s a *halting* n-state TM over $\{0, 1\}$ can write on an initially blank tape.

The Busy Beaver

BB grows faster than *any* computable function:

Example: $\mathrm{BB}(1) = 1, \mathrm{BB}(2) = 4, \mathrm{BB}(3) = 6, \mathrm{BB}(4) = 13, \mathrm{BB}(5) \geq 47\{,\}176\{,\}870.$

$\mathrm{BB}(6)$ is astronomically large (on the order of $10^{\left\{10^{\left\{10^{\{18705352\}}\right\}}\right\}}$).

BB is not computable. Suppose we could compute $\mathrm{BB}(n)$. Then to check whether an n-state TM M halts on blank tape: run M for $\mathrm{BB}(n)$ steps. If M hasn't halted by then, it never will. But this solves the Halting Problem — a contradiction.

Decidability

Decidable Sets

Definition 20: Given a universal set $\mathcal{U}$, a set $S \subseteq \mathcal{U}$ is *decidable* (or *computable*, or *recursive*) if there exists a computable function $f : \mathcal{U} \to \{0, 1\}$ such that $f(x) = 1$ iff $x \in S$.

Examples:

- The set of all WFFs is decidable.
  - *We can check if a given string is well-formed by recursively verifying the syntax rules.*
- For a given finite set Γ of WFFs, the set $\{\alpha \mid \Gamma \vDash \alpha\}$ of all tautological consequences of Γ is decidable.
  - *We can decide $\Gamma \vDash \alpha$ using a truth table algorithm by enumerating all possible interpretations (at most $2^{|\Gamma|}$) and checking if each satisfies all formulas in Γ.*
- The set of all tautologies is decidable.
  - *It is the set of all tautological consequences of the empty set.*

Undecidable Sets

Definition 21: A set S is *undecidable* if it is not decidable.

Example: The existence of *undecidable* sets can be shown as follows.

An algorithm is completely determined by its *finite* description. Thus, there are only *countably many* effective procedures. But there are *uncountably many* subsets of $\mathbb{N}$ (by Cantor's theorem). Hence, *most* sets of natural numbers are undecidable — decidable sets are the exception, not the rule.

FM implication: The set of *valid FOL formulas* is semi-decidable but not decidable (Church–Turing, 1936). This is why automated theorem provers for full FOL cannot be *complete deciders* — they can confirm validity but cannot always confirm invalidity.

Restricted theories (linear arithmetic, equality + uninterpreted functions) *are* decidable, which is exactly why SMT solvers work!

Undecidability

Halting Problem

Definition 22 (Halting problem): Given a program P and an input x, determine whether P *halts* on x (stops after finite time) or *loops* forever.

Theorem 2 (Turing, 1936): The halting problem is *undecidable*.

Proof *sketch*: Suppose there exists a procedure H that decides the halting problem. We can construct a program P that takes itself as input and runs H on it. If H says that P halts, then P enters an infinite loop. If H says that P does not halt, then P halts. This leads to a contradiction, proving that H cannot exist. $\square$

Halting Problem Pseudocode

```
def halts(P, x) -> bool:
  """
  Returns True if program P halts on input x.
  Returns False if P loops forever.
  """

def self_halts(P):
  if halts(P, P):
    while True: # loop forever
  else:
    return # halt
```

Observe that `halts(self_halts, self_halts)` cannot return neither `True` nor `False`. **Contradiction!**

Thus, the `halts` function *does not exist* (cannot be implemented), and the halting problem is *undecidable*.

Halting Problem Pseudocode [2]

Common confusion: The halting problem is *undecidable* for TMs (and all equivalent models). It does *not* mean we cannot detect simple loops in practice — a static analyzer can catch obvious infinite loops. It means there is *no algorithm* that correctly decides all possible programs.

Many-One Reductions

Definition 23: Language A is *many-one reducible* to language B, written $A \leq_m B$, if there exists a *total computable* function $f : \Sigma^* \to \Sigma^*$ such that for all $w \in \Sigma^*$:

$$w \in A \iff f(w) \in B$$

The function f is called the *reduction function*.

Theorem 3: If $A \leq_m B$ and B is decidable, then A is decidable.

Contrapositive (more useful): If $A \leq_m B$ and A is *undecidable*, then B is *undecidable*.

Strategy: To prove B is undecidable, show $\text{HALT} \leq_m B$ (or reduce from another problem A known to be undecidable).

Many-One Reductions [2]

Example: To prove $E_{\mathrm{TM}} = \{\langle M \rangle \mid \mathcal{L}(M) = \emptyset\}$ is undecidable:

Reduce HALT to the *complement* of E_{TM}: from $\langle M, w \rangle$, construct M' that ignores its own input and simulates M on w. Then M' accepts something iff M halts on w.

Since HALT is undecidable and the reduction is computable, E_{TM} is undecidable.

Rice's Theorem

Definition 24 (Semantic property): A property P of TMs is *semantic* if it depends only on the *language* recognized by the TM, not on the implementation.

Formally: if $\mathcal{L}(M_1) = \mathcal{L}(M_2)$ then $P(M_1) = P(M_2)$.

A property is *non-trivial* if *some* TMs satisfy it and *some* TMs do not.

Theorem 4 (Rice, 1953): Every non-trivial semantic property of Turing machines is *undecidable*.

That is, for any non-trivial semantic property P, the language L_P is undecidable, where:

$$L_P = \{\langle M \rangle \mid M \text{ has property } P\}$$

What Rice's theorem says: There is *no algorithm* that reads a TM (or program) description and correctly determines *any non-trivial language property* of what it computes.

Rice's Theorem — Proof

Proof: Let P be a non-trivial semantic property. WLOG assume P does not hold for the TM $M_\emptyset$ recognizing the empty language $\emptyset$. (If it does, use the complement of P instead.)

Since P is non-trivial, some TM M_P satisfies P, so $\mathcal{L}(M_P) \neq \emptyset$.

We reduce $A_{\text{TM}} = \{\langle M, w\rangle \mid M \text{ accepts } w\}$ to L_P. Given $\langle M, w\rangle$, construct a new machine M' as follows:

On input x:

1. Simulate M on w (ignoring x for now).
2. If M rejects w, **reject**.
3. If M accepts w, simulate M_P on x and output its result.

Then:

- If M accepts w: M' simulates M_P, so $\mathcal{L}(M') = \mathcal{L}(M_P)$, and $P(M')$ holds.
- If M does not accept w: M' never reaches step 3, so $\mathcal{L}(M') = \emptyset$, and $P(M')$ fails.

Thus $\langle M, w\rangle \in A_{\text{TM}} \iff \langle M'\rangle \in L_P$. Since A_{TM} is undecidable, so is L_P. $\square$ $\square$

Rice's Theorem — Consequences

No algorithm can decide, for an arbitrary program P:

- Does P terminate on *all* inputs?
- Does P ever produce output on *some* input?
- Are programs P_1 and P_2 *equivalent*?
- Does P recognize a *regular* language?
- Does P satisfy its specification?
- Does P have any security vulnerability?
- Does P produce infinite output?
- Does P access memory safely?

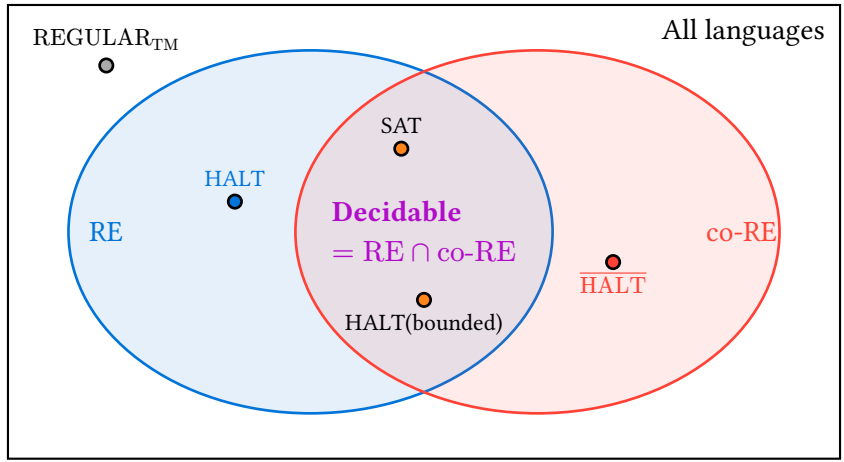
All of these are *non-trivial semantic properties* — all undecidable by Rice's theorem.

Automated *complete* program verification is *mathematically impossible* in full generality. This is not just an engineering obstacle — it is a *theorem*.

This is why Dafny requires *human-provided loop invariants* and pre/postconditions; why SMT solvers restrict to *decidable theories*; and why static analyzers produce *false positives*.

Formal verification is the art of finding the *right decidable fragment* for the problem at hand.

Map of Decidability



- **HALT** $\in$ RE $\setminus$ co-RE (recognizable but not co-recognizable).
- $\overline{\textbf{HALT}} \in$ co-RE $\setminus$ RE.
- **SAT** $\in$ R (decidable, NP-complete).
- **Bounded HALT** $\in$ R (decidable, co-NP-complete).
- **REGULAR$_{\text{TM}}$** — in neither RE nor co-RE!

Post Correspondence Problem

Definition 25 (Post correspondence problem): Given two finite lists $a_1, \ldots, a_n$ and $b_1, \ldots, b_n$ of strings (over the alphabet with at least two symbols), determine whether there exists a sequence of indices $i_1, \ldots, i_k$, such that $a_{i_1} \ldots a_{i_k} = b_{i_1} \ldots b_{i_k}$.

Example: Let $A = [a, ab, bba]$, $B = [baa, aa, bb]$. A solution is $(3, 2, 3, 1)$:

$$a_3 a_2 a_3 a_1 = bba \cdot ab \cdot bba \cdot a = bbaabbbaa = bb \cdot aa \cdot bb \cdot baa = b_3 b_2 b_3 b_1$$

An alternative formulation of PCP is a collection of *dominoes*, each with a *top* and a *bottom* half, with an unlimited supply of each block, and the goal is to find a sequence of blocks such that the string formed by the *top* halves is equal to the string formed by the *bottom* halves.

bba	ab	bba	a
bb	aa	bb	baa
$i_1 = 3$	$i_2 = 2$	$i_3 = 3$	$i_4 = 1$

Semi-decidability

Semi-decidability

Suppose we want to determine $\Sigma \vDash \alpha$, where Σ is infinite. In general, it is *undecidable*.

Definition 26 (Semi-decidable set): A set S is *computably enumerable* if there is an *enumeration procedure* which lists, in some order, every member of S: $s_1, s_2, s_3 \ldots$

Equivalently (see Theorem 5), a set S is *semi-decidable* if there is an algorithm such that the set of inputs for which the algorithm halts is exactly S.

Note: There are more synonyms for *computably enumerable*, such as *effectively enumerable*, *recursively enumerable* (do not confuse with just *recursive*!), and *Turing-recognizable*, or simply *recognizable*.

Note: If S is infinite, the enumeration procedure will *never* finish, but every member of S will be listed *eventually*, after some finite amount of time.

Note: Some properties of *decidable* and *semi-decidable* sets:

- Decidable sets are closed under union, intersection, Cartesian product, and complement.
- Semi-decidable sets are closed under union, intersection, and Cartesian product.

Enumerability and Semi-decidability

Theorem 5: A set S is computably enumerable iff it is semi-decidable.

Proof *($\Rightarrow$)*: *If S is computably enumerable, then it is semi-decidable.*

Since S is computably enumerable, we can check if $\alpha \in S$ by enumerating all members of S and checking if α is among them. If it is, we answer "yes"; otherwise, we continue enumerating. Thus, if $\alpha \in S$, the procedure produces "yes". If $\alpha \notin S$, the procedure runs forever. $\square$

Enumerability and Semi-decidability [2]

Proof *($\Leftarrow$): If S is semi-decidable, then it is computably enumerable.*

Suppose we have a procedure P which, given α, terminates and produces "yes" iff $\alpha \in S$. To show that S is computably enumerable, we can proceed as follows.

1. Construct a systematic enumeration of *all* expressions (for example, by listing all strings over the alphabet in length-lexicographical order): $\beta_1, \beta_2, \beta_3, \ldots$
2. Break the procedure P into a finite number of "steps" (for example, by program instructions).
3. Run the procedure on each expression in turn, for an increasing number of steps (see dovetailing):
   - Run P on β_1 for 1 step.
   - Run P on β_1 for 2 steps, then on β_2 for 2 steps.
   - ...
   - Run P on each of $\beta_1, \ldots, \beta_n$ for n steps each.
   - ...
4. If P produces "yes" for some β_i, output (yield) β_i and continue enumerating.

This procedure will eventually list all members of S, thus S is computably enumerable. $\square$

Dual Enumerability and Decidability

Theorem 6: A set is decidable iff both it and its complement are semi-decidable.

Proof *($\Rightarrow$)*: *If A is decidable, then both A and its complement $\overline{A}$ are effectively enumerable.*

Since A is decidable, there exists an effective procedure P that halts on all inputs and returs "yes" if $\alpha \in A$ and "no" otherwise.

To enumerate A:

- Systematically generate all expressions $\alpha_1, \alpha_2, \alpha_3, \ldots$
- For each α_i, run P. If P outputs "yes", yield α_i. Otherwise, continue.

Similarly, enumerate $\overline{A}$ by yielding α_i when P outputs "no".

Both enumerations are effective, since P always halts, so A and its complement are semi-decidable. $\square$

Dual Enumerability and Decidability [2]

Proof *($\Leftarrow$): If both A and its complement $\overline{A}$ are effectively enumerable, then A is decidable.*

Let E be an enumerator for A and $\overline{E}$ an enumerator for $\overline{A}$.

To decide if $\alpha \in A$, *interleave* the execution of E and $\overline{E}$, that is, for $n = 1, 2, 3, \ldots$

- Run E for n steps and if it produces α, *halt* and output "yes".
- Run $\overline{E}$ for n steps and if it produces α, *halt* and output "no".

Since α is either in A or in $\overline{A}$, one of the enumerators will eventually produce α. The interleaving with increasing number of steps ensures fair scheduling without starvation.

Remark: The "dovetailing" technique (alternating between enumerators with increasing step) avoids infinite waiting while maintaining finite memory requirements. The alternative is to run both enumerators *simultaneosly*, in parallel, using, for example, two computers. $\square$

Enumerability of Tautological Consequences

Theorem 7: If Σ is an effectively enumerable set of WFFs, then the set $\{\alpha \mid \Sigma \vDash \alpha\}$ of tautological consequences of Σ is effectively enumerable.

Proof: Consider an enumeration of the elements of Σ: $\sigma_1, \sigma_2, \sigma_3, \ldots$

By the compactness theorem, $\Sigma \vDash \alpha$ iff $\{\sigma_1, \ldots, \sigma_n\} \vDash \alpha$ for some n.

Hence, it is sufficient to successively test (using truth tables)

$$\begin{aligned} \emptyset &\vDash \alpha, \\ \{\sigma_1\} &\vDash \alpha, \\ \{\sigma_1, \sigma_2\} &\vDash \alpha, \end{aligned}$$

and so on. If any of these tests succeeds (each is decidable), then $\Sigma \vDash \alpha$.

This demonstrates that there is an effective procedure that, given any WFF α, will output "yes" iff α is a tautological consequence of Σ. Thus, the set of tautological consequences of Σ is effectively enumerable. $\square$

Universal Machines

Universal Turing Machine

A *universal Turing machine* is a Turing machine that is capable of computing any computable sequence. [1]

Definition 27: A *universal Turing machine* U_{TM} is a Turing machine that can simulate any other TM.

High-level description of a universal Turing machine U_{TM}:

- Given an input $\langle M, w \rangle$, where M is a TM and $w \in \Sigma^*$:
  - Run (simulate a computation of) M on w.
  - If M halts and accepts w, U_{TM} accepts $\langle M, w \rangle$.
  - If M halts and rejects w, U_{TM} rejects $\langle M, w \rangle$.
  - *Implicitly*, if M loops on w, U_{TM} loops on $\langle M, w \rangle$.

Definition 28: The *language of a universal Turing machine* U_{TM} is the set A_{TM} of all pairs (M, w) such that M is a TM and M accepts w.

$$A_{\mathrm{TM}} = \mathcal{L}(U_{\mathrm{TM}}) = \{\langle M, w \rangle \mid M \text{ is a TM and } w \in \mathcal{L}(M)\}$$

Diagonalization Language

Consider all possible Turing machines, listed in some order, and all strings that are valid TM descriptions:

$$\langle M_0 \rangle, \langle M_1 \rangle, \ldots$$

Definition 29: Construct the *diagonalization language* L_Δ of all TMs that do not accept their own description:

$$L_\Delta = \mathcal{L}(M_\Delta) = \{\langle M \rangle \mid M \text{ is a TM and } \langle M \rangle \notin \mathcal{L}(M)\}$$

Note: M_Δ is *not* listed in the table, since its behavior differs from each other M_i at least on input $\langle M_i \rangle$.

	$\langle M_0 \rangle$	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	…
M_0	Acc	No	No	Acc	No	…
M_1	Acc	Acc	Acc	Acc	Acc	…
M_2	Acc	Acc	No	No	No	…
M_3	No	Acc	Acc	Acc	Acc	…
M_4	No	Acc	No	No	No	…
$\vdots$	…	…	…	…	…	…
M_Δ	No	No	Acc	No	Acc	…

Diagonalization Language is not Recognizable

$L_\Delta = \{\langle M \rangle \mid \langle M \rangle \notin \mathcal{L}(M)\}$

Theorem 8: $L_\Delta \notin \mathrm{RE}$.

Proof: Suppose L_Δ is recognizable. Then there exists a recognizer R such that $\mathcal{L}(R) = L_\Delta$.

It is the case that either $\langle R \rangle \notin \mathcal{L}(R)$ or $\langle R \rangle \in \mathcal{L}(R)$.

1. $\langle R \rangle \notin \mathcal{L}(R)$. Thus, $\langle R \rangle \in L_\Delta$. Since $\mathcal{L}(R) = L_\Delta$, $\langle R \rangle \notin \mathcal{L}(R)$. Contradiction.
2. $\langle R \rangle \in \mathcal{L}(R)$. Thus, $\langle R \rangle \notin L_\Delta$. Since $\mathcal{L}(R) = L_\Delta$, $\langle R \rangle \in \mathcal{L}(R)$. Contradiction.

In either case, we reach a contradiction. Therefore, the initial assumption that L_Δ is recognizable must be false. Thus, L_Δ is not recognizable. $\square$

Universal Language

$A_{\text{TM}} = \mathcal{L}(U_{\text{TM}}) = \{\langle M, w\rangle \mid M \text{ is a TM and } w \in \mathcal{L}(M)\}$

Theorem 9: $A_{\text{TM}} \in \text{RE}$.

Proof: U_{TM} is a TM that recognizes A_{TM}. □

Theorem 10: $\overline{A}_{\text{TM}} \notin \text{RE}$

Proof: $L_\Delta \leq_M \overline{A}_{\text{TM}}$. Build a recognizer (impossible) for L_Δ using a (hypothetical) recognizer for $\overline{A}_{\text{TM}}$. □

Theorem 11: $A_{\text{TM}} \notin \text{R}$.

Proof: R is closed under complement. A language A is decidable iff it is both recognizable ($A \in \text{RE}$) and co-recognizable ($\overline{A} \in \text{RE}$). We know that $\overline{A}_{\text{TM}} \notin \text{RE}$, thus A_{TM} cannot be decidable. □

Reductions

Mapping Reductions

Many-one reductions are the primary tool for proving undecidability: show that if you could decide B, you could decide A (which is already known to be undecidable).

Definition 30 (Many-one reduction ($\leq_M$)): $A \leq_M B$ ("A reduces to B") means there is a computable total function f such that for all x: $x \in A \iff f(x) \in B$.

If $A \leq_M B$ and B is decidable, then A is decidable.

Contrapositive: If $A \leq_M B$ and A is undecidable, then B is undecidable.

Pattern for undecidability proofs:

1. Take a known undecidable problem A (*e.g.*, the Halting Problem $\mathrm{HALT}_{\mathrm{TM}}$).
2. Show $A \leq_M B$ by constructing f explicitly.
3. Conclude B is undecidable.

See the Undecidability section for worked examples: Halting Problem $\leq_M$ Rice's theorem problems.

Extremely Hard Problem

Regular languages are decidable. Some Turing machines accept regular languages and some do not.

Definition 31: Let **REGULAR** be the language of all TMs that accept regular languages.

$$\text{REGULAR}_{\text{TM}} = \{\langle M \rangle \mid \mathcal{L}(M) \text{ is regular}\}$$

This language is *neither* recognizable nor co-recognizable. (See theorems on the next slides.)

- No computer program can confirm that a given Turing machine has a *regular* language.
- No computer program can confirm that a given Turing machine has a *non-regular* language.

This problem is *beyond* the limits of what computers can *ever* do.

REGULAR is not Recognizable

Theorem 12: $\mathrm{REGULAR}_{\mathrm{TM}} \notin \mathrm{RE}$.

Proof: $L_\Delta \leq_M \mathrm{REGULAR}_{\mathrm{TM}}$. $\square$

REGULAR is not even co-Recognizable

Theorem 13: $\text{REGULAR}_{\text{TM}} \notin \text{co-RE}$

Proof: $\overline{L}_{\Delta} \leq_M \text{REGULAR}_{\text{TM}}$. $\square$

Alternative Models of Computation

Alternative Models

Beyond Turing machines, every general-purpose model computes the same class of functions:

Model	Author (year)
λ-calculus	Church (1936)
μ-recursive functions	Kleene (1936)
Post correspondence systems	Post (1943)
Markov algorithms	Markov (1951)
Register machines (RAM)	Shepherdson & Sturgis (1963)
Any general-purpose language	(present day)

Historical note: Church and Turing independently proved these models equivalent in 1936.

Their results also resolved Hilbert's Entscheidungsproblem: FOL validity is *not* decidable.

λ-Calculus in a Nutshell

The λ-calculus is a minimal language with just three constructs:

Definition 32 (λ-Calculus Syntax):

$$M ::= x \mid (\lambda x.M) \mid (M\ N)$$

Variables, abstraction (function definition), and application (function call).

Example:

- *Identity:* $\lambda x.x$ — takes x, returns x.
- *Application:* $(\lambda x.x)\ y \rightsquigarrow y$ — β-reduction.
- *Church numeral* $\overline{2}$: $\lambda f.\lambda x.f(f(x))$ — "apply f twice".

Computation is *β-reduction*: $(\lambda x.M)\ N \rightsquigarrow M[x := N]$ (substitute N for x in M).

Key insight: Despite having no numbers, booleans, or loops, λ-calculus can encode *all* computable functions. This is the theoretical foundation of functional programming (Haskell, ML, Coq, Lean).

From Theory to Practice

Bridging Computability and Software Engineering

Theory says:

- Program correctness is undecidable (Rice).
- Halting is undecidable.
- FOL validity is undecidable.
- Full functional equivalence is undecidable.

Practice responds:

- Sound approximation (abstract interpretation).
- Decidable fragments (SMT theories).
- Programmer annotations (Dafny, ACSL, JML).
- Bounded verification (SAT, BMC, k-induction).

Example: Dafny's approach combines decidable theories (linear arithmetic, arrays, sets) with programmer-supplied loop invariants and pre/postconditions to make verification *tractable* for real programs.

The key message: Undecidability is not a dead end — it is a *design constraint*. Formal methods succeed by carefully choosing *what* to verify and *how much* automation to provide.

Looking Ahead

Week 3: SAT
- NP-completeness
- CDCL solvers
- SAT encodings

Week 6: SMT
- Decidable theories
- DPLL(T) architecture
- Theory combination

Weeks 9–12: Dafny
- Annotations
- Loop invariants
- Verified programs

Each step makes the *gap between theory and practice* narrower: from "undecidable in general" to "verified for this specific program".

Exercises

Exercises: Decidability and Computability

1. Show that the language $\{\langle M \rangle \mid M \text{ accepts at least one string}\}$ is recognizable but not decidable.

2. Using Rice's theorem, explain why each of the following is undecidable:
   - $\{\langle M \rangle \mid \mathcal{L}(M) = \Sigma^*\}$ (universality)
   - $\{\langle M \rangle \mid \mathcal{L}(M) \text{ is context-free}\}$
   - $\{\langle M \rangle \mid |\mathcal{L}(M)| = 42\}$

3. Construct a reduction from $\mathrm{HALT}_{\mathrm{TM}}$ to $\mathrm{TOTAL}_{\mathrm{TM}} = \{\langle M \rangle \mid M \text{ halts on all inputs}\}$ to prove $\mathrm{TOTAL}_{\mathrm{TM}}$ is undecidable.

4. Explain why Rice's theorem does *not* apply to the property "M has fewer than 10 states." What kind of property is this?

5. $\star$ The *Busy Beaver* function $\mathrm{BB}(n)$ = the maximum number of steps any halting TM with n states can make. Argue that BB grows faster than any computable function. *Hint:* if BB were computable, we could decide the halting problem.

6. $\star$ Consider λ-terms $I = \lambda x.x$ and $\Omega = (\lambda x.x\ x)(\lambda x.x\ x)$.
   Show that I has a normal form, but Ω does not (*i.e.*, β-reduction does not terminate on Ω).

Bibliography

[1] A. M. Turing, "On Computable Numbers, with an Application to the Entscheidungsproblem," *Proceedings of the London Mathematical Society*, no. 1, pp. 230–265, 1937, doi: 10.1112/plms/s2-42.1.230.